Simple Linear Regression — Gradient Descent

A — Loss & Cost Function

Loss (single point): $$e_i = (y_i - \hat{y}_i)^2$$

Cost (MSE over n points): $$f = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

B — Prediction Line

Predicted value: $$\hat{y}_i = \hat{m}\,x_i + \hat{b}$$

C — Cost Function (expanded)

Substituting prediction: $$f = \frac{1}{n}\sum_{i=1}^{n}\left[y_i - (\hat{m}\,x_i + \hat{b})\right]^2$$

D — Partial Derivatives (Gradients)

w.r.t. slope m : $$\frac{\partial f}{\partial m} = -\frac{2}{n}\sum_{i=1}^{n} x_i\left[y_i - (\hat{m}\,x_i + \hat{b})\right]$$

w.r.t. intercept b : $$\frac{\partial f}{\partial b} = -\frac{2}{n}\sum_{i=1}^{n}\left[y_i - (\hat{m}\,x_i + \hat{b})\right]$$

Gradient vector: $$\nabla f = \left[\frac{\partial f}{\partial m}, \frac{\partial f}{\partial b}\right]^T$$

E — Parameter Update Rules

Update slope: $$m_{new} = m - \lambda\frac{\partial f}{\partial m}$$

Update intercept: $$b_{new} = b - \lambda\frac{\partial f}{\partial b}$$

F — Optimisation Goal

Find m, b that minimise: $$\arg\min_{m,\,b}\frac{1}{n}\sum_{i=1}^{n}\left[y_i - (\hat{m}\,x_i + \hat{b})\right]^2$$

Symbol Legend

y_i	$\hat{y}_i$	m	b	λ	n
actual value	predicted value	slope	intercept	learning rate	no. of samples

Solved with Gradient Descent | $\lambda = 0.01635$ | $m_0 = 3.7426$ | $b_0 = 54.8732$

Gradient Descent — Algorithm Flowchart

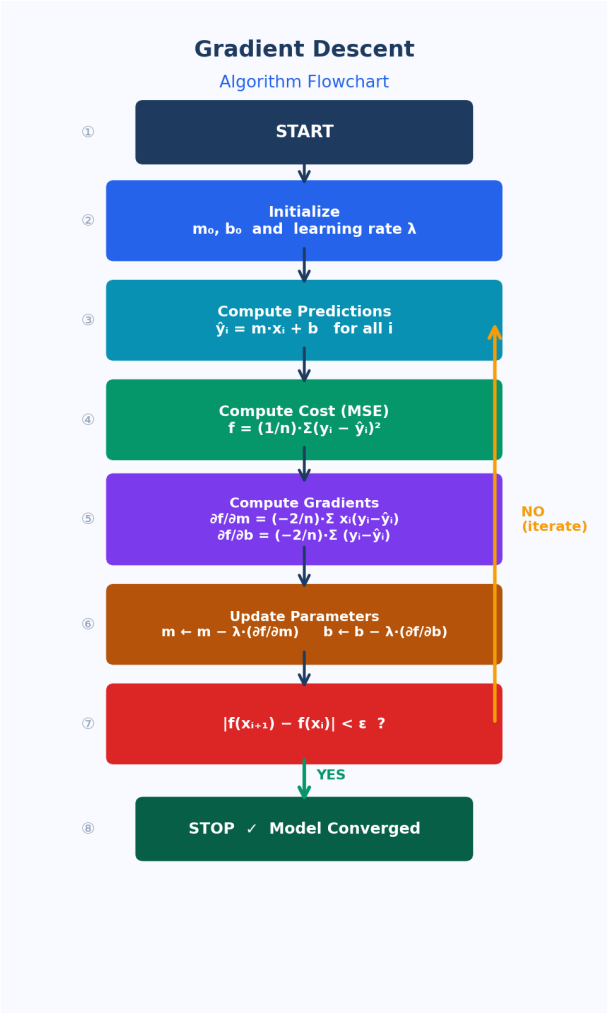

Figure 1 — Step-by-step gradient descent loop with convergence check

Numerical Solution

Given Starting Values

Parameter	Symbol	Value
Initial slope	m■	3.7426
Initial intercept	b■	54.8732
Learning rate	λ	0.01635
Number of samples	n	10

Iteration 1 — Using m■ = 3.7426, b■ = 54.8732

x	y	■ = m·x + b	y – ■	x · (y – ■)
2	69	62.3584	6.6416	13.2832
9	98	88.5566	9.4434	84.9906
5	82	73.5862	8.4138	42.0690
5	77	73.5862	3.4138	17.0690
3	71	66.1010	4.8990	14.6970
7	84	81.0714	2.9286	20.5002
1	55	58.6158	-3.6158	-3.6158
8	94	84.8140	9.1860	73.4880
6	84	77.3288	6.6712	40.0272
2	64	62.3584	1.6416	3.2832
	Σ		**49.6232**	**305.7916**

```
∂f/∂m = (–2/10) × 305.7916 = -61.1583
```

```
∂f/∂b = (–2/10) × 49.6232 = -9.9246
```

```
m■ = 3.7426 – 0.01635 × (-61.1583) = 4.742539
```

```
b■ = 54.8732 – 0.01635 × (-9.9246) = 55.035468
```

✔ MSE after Iteration 1 = **7.9121** (was 39.2968 → reduced by 31.38)

Iteration 2 — Using m■ = 4.742539, b■ = 55.035468

x	y	■ = m·x + b	y – ■	x · (y – ■)
2	69	64.5205	4.4795	8.9589
9	98	97.7183	0.2817	2.5352
5	82	78.7482	3.2518	16.2592
5	77	78.7482	-1.7482	-8.7408
3	71	69.2631	1.7369	5.2107
7	84	88.2332	-4.2332	-29.6327
1	55	59.7780	-4.7780	-4.7780
8	94	92.9758	1.0242	8.1938
6	84	83.4907	0.5093	3.0558
2	64	64.5205	-0.5205	-1.0411
	Σ		**0.0035**	**0.0211**

```
∂f/∂m = (–2/10) × 0.021060 ≈ -0.004212 (near zero!)
```

```
∂f/∂b = (–2/10) × 0.003472 ≈ -0.000694 (near zero!)
```

```
m■ = 4.742539 – 0.01635 × (-0.004212) = 4.742607
```

```
b■ = 55.035468 – 0.01635 × (-0.000694) = 55.035479
```

✔ MSE after Iteration 2 = **7.9121** — model has converged!

Results Summary

	Slope m	Intercept b	MSE Cost	Cost Drop
Iteration 0	3.7426	54.8732	39.2968	—
Iteration 1	4.742539	55.035468	7.9121	↓ 31.38
Iteration 2	**4.742607**	**55.035479**	**7.9121**	**↓ 0.0000**

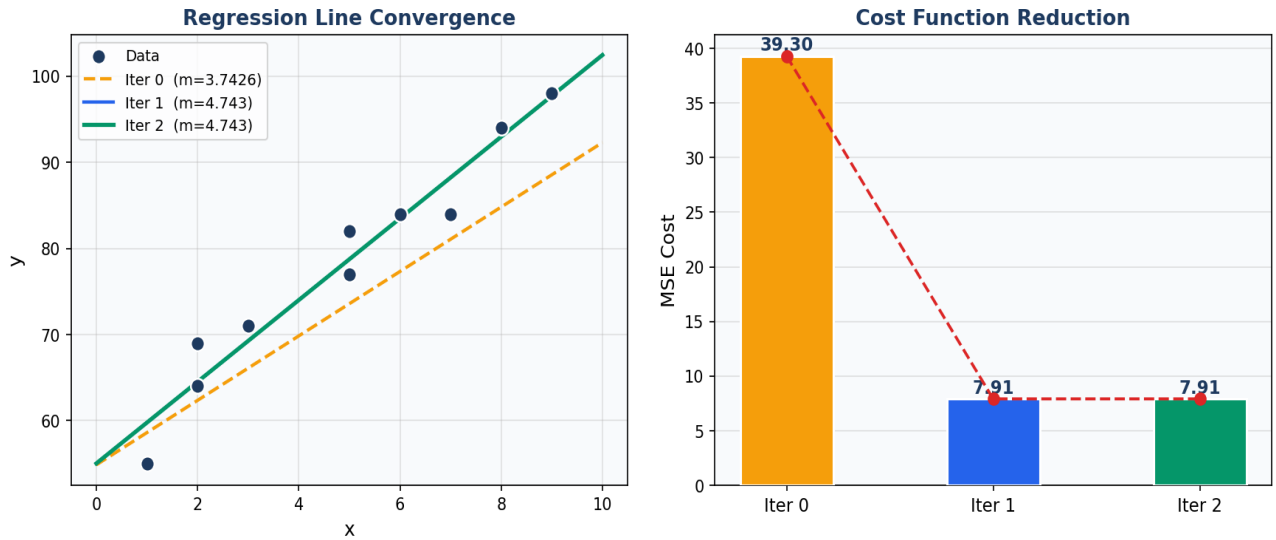

Figure 2 — Left: Regression line converging over iterations | Right: MSE cost dropping

Key Takeaways

- Gradient descent converged in just **2 iterations** for this dataset.
- MSE cost dropped from **39.30** → **7.91** → **7.9121**.
- Final fitted line: **■ = 4.7426 · x + 55.0355**
- Gradients became nearly zero (≈ 0.000004) — no more meaningful update.
- A smaller λ → more iterations; larger λ → risk of overshooting the minimum.